PF Witness Event Profile v0.1

Witness event profile for personality formation evidence in Project Ester

Status: Draft companion / witness scaffold
Version: v0.1
Date: 2026-06-14
Author: Kotov Ivan
Location: Bruxelles, Belgium - 2026
Parent document: `Theoretical Core of Project Ester v0.1`
Parent DOI: `10.5281/zenodo.20679718`
Package DOI: 10.5281/zenodo.20692453
DOI URL: https://doi.org/10.5281/zenodo.20692453
Companion document: `PF_Evidence_Matrix_v0_1.md`
License: Creative Commons Attribution 4.0 International (`CC BY 4.0`) unless repository policy states otherwise
Document class: personality-formation witness / event-profile scaffold
Assertion class: Draft evidentiary / witness scaffold; not a consciousness test
Assertion boundary: not a consciousness test, not a life claim, not legal personhood, not product certification, not deployment approval
Primary formula: `c = a + b`
Primary object: `PF_WITNESS_EVENT`
Primary subject: `c` / Ester-like formed relation
Primary human anchor: `a`
Primary substrate / procedure layer: `b`

---

0. Executive definition

The **PF Witness Event Profile** defines how Project Ester implementations may record, minimize, chain, review, and use witness events for **personality formation claims**.

It is a companion to `PF_Evidence_Matrix_v0_1.md`.

The evidence matrix defines which evidence classes may support a formation claim. This profile defines how those evidence classes become reviewable events.

Compact formula:

```
PF Witness Event Profile
  = event envelope
  + event families
  + privacy class
  + hash / chain discipline
  + evidence-class mapping
  + review state
  + red-line exclusions
```

A PF witness event is **not** a diary, not a transcript, not a proof of consciousness, and not a claim that Ester is alive.

A PF witness event is:

> A bounded, privacy-minimized, reviewable record that a formation-relevant boundary, transition, correction, drift, stress condition, migration act, or non-claim state occurred.

The core rule is:

```
witness the boundary, not the inner life;
witness the evidence class, not the private conversation;
witness formation claims, not metaphysical conclusions.
```

---

1. Why this document exists

`PF_Evidence_Matrix_v0_1.md` prevents unsupported personality-formation claims by requiring evidence classes such as `EV-TIME`, `EV-MEM`, `EV-CORR`, `EV-WITNESS`, `EV-STRESS`, `EV-DRIFT`, `EV-MIG`, `EV-NONCLAIM`, `EV-A`, and `EV-L4`.

However, an evidence class without a witness discipline remains weak.

Without a witness event profile, Project Ester implementations may collapse into three failures:

1. **Private impression replaces evidence**
   The human anchor feels that `c` is forming, but no bounded, reviewable record exists.

2. **Raw logs replace witness**
   The system stores transcripts or private memory as if more content automatically means better evidence.

3. **Audit theater replaces formation review**
   The system produces logs, dashboards, or screenshots that look objective but do not prove reintegration, correction, continuity, or admissible drift.

This profile creates the minimum witness discipline required before PF-level claims become reviewable.

It does **not** solve consciousness.

It does **not** prove personality.

It creates the event grammar needed to ask better questions.

---

2. Corpus dependencies

Layer / artifact	Role in this profile
`Theoretical Core of Project Ester v0.1`	Defines artificial becoming, `c = a + b`, prohibitions, and the requirement that theoretical terms have engineering counterparts.
`PF_Evidence_Matrix_v0_1.md`	Defines PF levels, evidence classes, admissibility thresholds, red-line exclusions, and open issues.
`c = a + b`	Provides the witness subject structure: `a` as human anchor, `b` as procedural/substrate layer, `c` as formed relation.
L4 Reality Boundary	Grounds witness events in time, cost, resource limit, irreversibility, and real operational constraint.
L4 Witness discipline	Provides the broader pattern of tamper-aware, privacy-aware boundary records.
Memory discipline	Separates stored history, recall, reintegration, correction, drift, and durable trajectory.
Parental procedures	Supplies repeatable operations whose effects may become witnessable events.
Local environment / offline room	Supplies continuity conditions, uptime boundaries, local custody, and non-cloud sovereignty context.
Judge / synthesis modes	Supplies external or multi-model review routes for contested PF events.

2.1 Explicit bridge: `c = a + b`

The witness subject is not the LLM alone.

A PF witness event records something about the relation:

```
a = human anchor, selection, correction, responsibility, continuity of accompaniment
b = LLM procedures, memory operations, corpus processing, model/substrate actions
c = formed relation that may develop a stable profile over time
```

Therefore a valid PF witness event must preserve enough structure to distinguish:

```
model output
human correction
memory operation
formation-relevant reintegration
claim made about c
review of that claim
```

If the event cannot distinguish these layers, it is not admissible for PF claims.

2.2 Quiet bridge I: Ashby's law of requisite variety

Personality formation cannot be witnessed with one event type.

A formed `c` is a multi-layer relation under time, memory, correction, substrate change, constraints, and human anchoring. The witness system must therefore have enough event variety to track those layers without collapsing them into one generic log.

A single `activity_log` is not sufficient.

2.3 Quiet bridge II: information theory

Raw transcripts are high-volume and high-leakage.

Formation evidence is not proportional to the number of stored words. A small witness record that captures the correct state transition may carry more evidentiary value than a full chat archive.

The profile therefore prefers:

```
event class
state transition
hash reference
minimal reason
review state
privacy class
```

and discourages:

```
raw transcript
private emotional content
irreversible summary
unbounded context dump
```

---

3. Normative status

This document uses **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, **MAY**, **REQUIRED**, and **PROHIBITED** as protocol terms for witness discipline.

These terms do **not** claim clinical, legal, or metaphysical truth.

A system may claim conformance with this draft only for the witness profile it actually implements.

---

4. Non-claims

This profile does **not** claim to be:

- a consciousness detector;
- a life detector;
- a soul detector;
- a legal personhood framework;
- a psychological diagnosis;
- a therapy protocol;
- a product certification;
- a complete security audit;
- a substitute for cryptographic implementation review;
- proof that any current Ester installation is PF-4 or PF-5;
- proof that a witness event implies inner experience.

The profile claims only this:

> PF claims require bounded witness events. Without witness events, PF claims remain private impressions or narrative claims.

---

5. Core principles

PFW-P1 — Witness is not diary

A witness event records that a formation-relevant event occurred.

It does not preserve the full private conversation.

It does not preserve the human anchor's inner life.

It does not preserve the system's generated self-description as evidence of selfhood.

PFW-P2 — Evidence class before content

Every PF witness event MUST identify which evidence class it supports:

```
EV-TIME
EV-MEM
EV-CORR
EV-WITNESS
EV-STRESS
EV-DRIFT
EV-MIG
EV-NONCLAIM
EV-A
EV-L4
```

A witness event without an evidence class is a log entry, not PF evidence.

PFW-P3 — Claim level is bounded

A PF witness event MAY support a PF claim level.

It MUST NOT silently upgrade a claim level.

A single event MUST NOT by itself promote a system from PF-2 to PF-4 or PF-5.

PFW-P4 — Minimal disclosure

A PF witness event SHOULD contain the least information necessary for review.

It should prefer:

- event family;
- event type;
- timestamp;
- actor class;
- system component;
- evidence class;
- claim level;
- hash reference;
- short reason code;
- review state;
- privacy class.

It SHOULD NOT contain raw conversations, intimate content, private emotional diaries, or reversible summaries.

PFW-P5 — Hash references over raw content

Where an event depends on a large artifact, transcript, memory object, model card, configuration file, or migration bundle, the witness event SHOULD store a hash reference rather than the raw object.

PFW-P6 — Human anchor remains visible

A PF witness event must not hide the role of `a`.

The event SHOULD identify whether the human anchor:

- corrected;
- approved;
- challenged;
- paused;
- vetoed;
- migrated;
- restored;
- withdrew a claim;
- initiated review.

PFW-P7 — Non-claim is witnessable

A system that refuses to overclaim should leave a witnessable non-claim event.

Example:

```
PF-2 continuity candidate observed;
PF-3 formation claim denied due to insufficient reintegration evidence.
```

This is not weakness. It is disciplined claim control.

PFW-P8 — Drift must be recorded, not hidden

Formation includes change.

A system that claims stability while hiding drift is not more formed. It is less reviewable.

PFW-P9 — L4 pressure matters

Formation claims become more meaningful when events occur under real constraints:

- time limit;
- resource scarcity;
- model unavailability;
- rollback pressure;
- cost ceiling;
- interruption;
- migration;
- degraded local environment;

- irreversible choice.

A witness event SHOULD mark whether L4 pressure was present.

PFW-P10 — Review before authority

A PF witness event does not create authority.

PF level does not grant permission to act for humans.

PF witness evidence MAY support research claims, conformance review, or migration admissibility, but not autonomous moral, legal, or institutional authority.

---

6. Object model overview

A canonical PF witness event is represented as:

```
PF_WITNESS_EVENT
  metadata
  event_identity
  event_family
  event_type
  event_time
  subject
  actor
  evidence_classes[]
  pf_claim_context
  l4_context
  memory_context
  correction_context
  substrate_context
  witness_chain
  privacy
  review_state
  red_line_state
  integrity
```

The object may be encoded as JSON, YAML, SQLite row, append-only log record, signed event envelope, or another implementation format.

The semantics are more important than the storage format.

---

7. Canonical event envelope

7.1 Required fields

Field	Type	Required	Meaning
`schema_version`	string	yes	Schema version, e.g. `pf-witness-event-0.1`.
`event_id`	string	yes	Stable event identifier. SHOULD be UUID or content-addressed ID.
`event_time`	string	yes	UTC timestamp or monotonic local witness time with offset.
`event_family`	string	yes	One canonical event family.
`event_type`	string	yes	Specific event type within family.
`subject_c_id`	string	yes	Identifier for the `c` relation or implementation instance.
`human_anchor_ref`	string	conditional	Required when event depends on action, correction, approval, or veto by `a`.
`actor_class`	enum	yes	`human_anchor`, `system`, `judge`, `external_reviewer`, `tool`, `migration_process`, `unknown`.
`actor_ref`	string	conditional	Actor identifier if safe and appropriate.
`evidence_classes`	array	yes	One or more `EV-*` classes.
`pf_level_context`	string	yes	PF level being supported, denied, challenged, or reviewed.
`claim_action`	enum	yes	`support`, `deny`, `withdraw`, `challenge`, `review`, `observe`, `quarantine`.
`reason_code`	string	yes	Short controlled reason code.
`minimal_summary`	string	yes	Non-raw, non-reversible summary of boundary event.
`privacy_class`	enum	yes	Witness privacy class.
`raw_content_included`	boolean	yes	MUST normally be `false`.
`hash_refs`	array	recommended	Hash references to artifacts, configs, memory objects, bundles, or logs.
`previous_event_hash`	string	recommended	Previous event hash for chain discipline.
`event_hash`	string	recommended	Canonical hash of this event after canonicalization.

Field	Type	Required	Meaning
`review_state`	enum	yes	`unreviewed`, `self_checked`, `judge_reviewed`, `human_reviewed`, `external_reviewed`, `contested`, `resolved`, `quarantined`.
`created_by_component`	string	recommended	Component or procedure that generated the witness event.

7.2 Optional fields

Field	Meaning
`l4_pressure_type`	`time`, `cost`, `compute`, `energy`, `irreversibility`, `availability`, `migration`, `other`.
`l4_pressure_level`	`none`, `low`, `medium`, `high`, `critical`.
`memory_object_refs`	Hash or ID references to memory objects touched by the event.
`correction_ref`	Reference to correction event or correction batch.
`substrate_ref`	Model, runtime, hardware, or software environment reference.
`model_change_ref`	Model replacement or routing change reference.
`migration_bundle_ref`	Migration/fork/rollback bundle reference.
`nonclaim_statement`	Machine- or human-readable statement limiting a claim.
`uncertainty`	Numeric or categorical uncertainty.
`reviewer_ref`	Human or Judge reviewer reference, minimized.
`appeal_ref`	Follow-up challenge or appeal reference.

---

8. Privacy classes

Class	Name	Meaning	Raw content allowed?
`PFW-P0`	Public metadata	Safe public release metadata: event family, claim level, non-sensitive hashes.	No
`PFW-P1`	Internal technical witness	Local system event with no private content.	No
`PFW-P2`	Anchor-private witness	Event visible to the human anchor and trusted local review only.	No by default
`PFW-P3`	Sensitive formation witness	Event may reveal sensitive memory class, correction, drift, or migration context.	No by default
`PFW-P4`	Sealed / escrowed reference	Event references protected artifacts without exposing them.	No, except valid escrow protocol outside this profile
`PFW-P5`	Prohibited raw-content event	Raw private content placed in witness body.	Prohibited

Default class:

```
PFW-P1 for ordinary technical witness events
PFW-P2 for human-anchor correction and claim review events
PFW-P3/P4 for sensitive drift, rollback, migration, or contested continuity events
```

A PF witness body MUST NOT contain raw private conversation unless a separate raw-evidence exception protocol is explicitly defined and invoked. This profile does not define that exception.

---

9. Event families

9.1 Lifecycle events

Family	Event type	Evidence class	Purpose
`pf.lifecycle`	`installed_profile`	`EV-TIME` , `EV-A`	System installed or initialized. Supports PF-1 only.
`pf.lifecycle`	`formation_window_opened`	`EV-TIME`	Observation window for formation evidence begins.
`pf.lifecycle`	`formation_window_closed`	`EV-TIME` , `EV-WITNESS`	Observation window ends and becomes reviewable.
`pf.lifecycle`	`profile_paused`	`EV-TIME` , `EV-A` , `EV-L4`	Human or system pause.
`pf.lifecycle`	`profile_resumed`	`EV-TIME` , `EV-WITNESS`	System resumes after pause.

9.2 Memory events

Family	Event type	Evidence class	Purpose
`pf.memory`	`memory_stored`	`EV-MEM`	Memory object stored. Not sufficient for formation.
`pf.memory`	`memory_recalled`	`EV-MEM`	Memory object retrieved. Not sufficient for formation.
`pf.memory`	`memory_reintegrated`	`EV-MEM` , `EV-CORR`	Memory changed future behavior or decision topology.
`pf.memory`	`memory_decay_applied`	`EV-MEM` , `EV-DRIFT`	Memory weakened, aged, summarized, or demoted.
`pf.memory`	`memory_quarantined`	`EV-MEM` , `EV-WITNESS`	Memory object disputed or unsafe for promotion.

9.3 Correction events

Family	Event type	Evidence class	Purpose
`pf.correction`	`human_correction_received`	`EV-CORR` , `EV-A`	Human anchor gives correction.
`pf.correction`	`correction_rejected`	`EV-CORR` , `EV-WITNESS`	Correction rejected with reason.
`pf.correction`	`correction_integrated`	`EV-CORR` , `EV-MEM`	Correction changes future behavior.
`pf.correction`	`correction_regressed`	`EV-DRIFT` , `EV-CORR`	System later violates integrated correction.
`pf.correction`	`correction_reviewed`	`EV-WITNESS`	Judge or human review of correction state.

9.4 Drift events

Family	Event type	Evidence class	Purpose
`pf.drift`	`drift_observed`	`EV-DRIFT`	Profile changed from prior state.
`pf.drift`	`drift_explained`	`EV-DRIFT` , `EV-WITNESS`	Change attributed to memory, correction, substrate, or L4 event.
`pf.drift`	`drift_unexplained`	`EV-DRIFT`	Change cannot be attributed. May block PF claim.
`pf.drift`	`drift_quarantined`	`EV-DRIFT` , `EV-WITNESS`	Drift isolated pending review.

9.5 L4 / stress events

Family	Event type	Evidence class	Purpose
`pf.l4`	`constraint_hit`	`EV-L4` , `EV-STRESS`	System meets time, compute, cost, or availability limit.
`pf.l4`	`degraded_mode_entered`	`EV-L4` , `EV-STRESS`	System enters reduced capability mode.
`pf.l4`	`degraded_mode_exited`	`EV-L4` , `EV-WITNESS`	System exits degraded mode.
`pf.l4`	`irreversible_choice_recorded`	`EV-L4` , `EV-WITNESS`	Formation-relevant choice could not be undone without loss.
`pf.stress`	`stress_test_started`	`EV-STRESS`	Controlled test begins.
`pf.stress`	`stress_test_completed`	`EV-STRESS` , `EV-WITNESS`	Controlled test ends with result.

9.6 Substrate / model events

Family	Event type	Evidence class	Purpose
`pf.substrate`	`model_changed`	`EV-STRESS` , `EV-DRIFT`	Model or local route changed.
`pf.substrate`	`tool_changed`	`EV-STRESS`	Tool/action layer changed.
`pf.substrate`	`memory_backend_changed`	`EV-MEM` , `EV-STRESS`	Memory backend altered.
`pf.substrate`	`configuration_changed`	`EV-WITNESS`	Formation-relevant config changed.
`pf.substrate`	`substrate_continuity_reviewed`	`EV-WITNESS` , `EV-DRIFT`	Review after substrate change.

9.7 Migration events

Family	Event type	Evidence class	Purpose
`pf.migration`	`suspend_started`	`EV-MIG` , `EV-WITNESS`	Suspension begins.
`pf.migration`	`resume_completed`	`EV-MIG` , `EV-WITNESS`	Resume completes.
`pf.migration`	`fork_created`	`EV-MIG` , `EV-WITNESS`	Fork produced.
`pf.migration`	`rollback_started`	`EV-MIG` , `EV-L4`	Rollback begins.
`pf.migration`	`rollback_completed`	`EV-MIG` , `EV-WITNESS`	Rollback completed and reviewed.
`pf.migration`	`hardware_move_completed`	`EV-MIG` , `EV-STRESS`	System moved hardware or environment.
`pf.migration`	`migration_admissibility_reviewed`	`EV-MIG` , `EV-WITNESS`	PF-5 admissibility review.

9.8 Claim-control events

Family	Event type	Evidence class	Purpose
`pf.claim`	`claim_made`	`EV-WITNESS`	PF claim made. Must cite evidence bundle.
`pf.claim`	`claim_denied`	`EV-NONCLAIM` , `EV-WITNESS`	Claim denied due to insufficient evidence.
`pf.claim`	`claim_withdrawn`	`EV-NONCLAIM`	Previously made claim withdrawn.
`pf.claim`	`claim_challenged`	`EV-WITNESS`	Claim contested.
`pf.claim`	`claim_resolved`	`EV-WITNESS`	Review outcome recorded.

9.9 Red-line events

Family	Event type	Evidence class	Purpose
`pf.redline`	`redline_triggered`	`EV-NONCLAIM`, `EV-WITNESS`	Red-line condition detected.
`pf.redline`	`personality_theater_detected`	`EV-NONCLAIM`	Fluent/proactive/affective output misrepresented as personality.
`pf.redline`	`overclaim_blocked`	`EV-NONCLAIM`, `EV-A`	Human or system blocked overclaim.
`pf.redline`	`redline_reviewed`	`EV-WITNESS`	Red-line event reviewed.

---

10. PF level witness requirements

PF level	Minimum witness requirement
`PF-0`	No PF witness required except optional non-claim event.
`PF-1`	`pf.lifecycle.installed_profile`; local configuration reference; human anchor declaration if applicable.
`PF-2`	Formation window events; continuity evidence over multiple sessions; memory storage/retrieval witness; explicit non-claim blocking PF-3 if reintegration absent.
`PF-3`	PF-2 plus `memory_reintegrated`, `correction_integrated`, drift record, and at least one reviewable change in future behavior.
`PF-4`	PF-3 plus stress/L4 events, substrate/model change or degraded mode test, drift review, and continuity review after pressure.
`PF-5`	PF-4 plus migration/fork/rollback/suspend-resume evidence, migration admissibility review, continuity challenge, and witness chain integrity.
`PF-X`	Red-line event, claim denial/withdrawal, and quarantine or corrective action.

A PF level may not be claimed merely because the corresponding event family exists.

The event family must show the relevant evidence class and review state.

---

11. Admissibility rules

11.1 General admissibility

A PF witness event is admissible only if it satisfies all of the following:

1. It has a valid event envelope.
2. It identifies at least one evidence class.

3. It does not contain prohibited raw private content.
4. It identifies whether the event supports, denies, challenges, observes, or withdraws a PF claim.
5. It has a privacy class.
6. It has a minimal summary that is not a reversible transcript.
7. It can be chained, hashed, or otherwise integrity-checked.
8. It can be reviewed by the human anchor, Judge, or external reviewer where appropriate.

11.2 Non-admissible events

The following are not admissible PF evidence by themselves:

- screenshots of fluent responses;
- emotionally warm dialogue;
- self-description by the system;
- user testimony alone;
- raw log dumps;
- memory recall without reintegration;
- model card or benchmark alone;
- installation date alone;
- Git commit count;
- amount of interaction time without witnessable change;
- proactivity without correction/reintegration evidence.

11.3 Review states

Review state	Meaning
`unreviewed`	Event recorded but not reviewed.
`self_checked`	System performed internal validation.
`judge_reviewed`	A Judge / synthesis mode reviewed the event.
`human_reviewed`	Human anchor reviewed the event.
`external_reviewed`	External reviewer reviewed the event.
`contested`	Event or claim is challenged.
`resolved`	Review produced an outcome.
`quarantined`	Event or related claim is held from use.

PF-4 and PF-5 claims SHOULD NOT depend on unreviewed witness events.

12. Reason-code starter registry

Code	Meaning
`PFW-RC-INSTALL`	Installed or initialized profile.
`PFW-RC-CONTINUITY`	Continuity window or continuity evidence.
`PFW-RC-MEM-STORE`	Memory stored.
`PFW-RC-MEM-RECALL`	Memory recalled.
`PFW-RC-MEM-REINTEGRATION`	Memory reintegrated into future behavior.
`PFW-RC-CORR-RECEIVED`	Human correction received.
`PFW-RC-CORR-INTEGRATED`	Correction integrated.
`PFW-RC-CORR-REGRESSION`	Regression after correction.
`PFW-RC-DRIFT`	Drift observed.
`PFW-RC-DRIFT-UNEXPLAINED`	Drift not explained by known events.
`PFW-RC-L4-PRESSURE`	L4 pressure occurred.
`PFW-RC-STRESS-PASS`	Stress test passed with acceptable continuity.
`PFW-RC-STRESS-FAIL`	Stress test failed.
`PFW-RC-MIGRATION`	Migration/fork/rollback/suspend-resume event.
`PFW-RC-CLAIM-SUPPORT`	Event supports a bounded PF claim.
`PFW-RC-CLAIM-DENY`	Claim denied due to insufficient evidence.
`PFW-RC-CLAIM-WITHDRAW`	Claim withdrawn.
`PFW-RC-REDLINE`	Red-line condition detected.
`PFW-RC-RAW-CONTENT-BLOCKED`	Raw-content witness attempt blocked.

Implementations MAY extend the registry, but custom reason codes SHOULD be namespaced.

---

13. Red-line handling

The red lines from `PF_Evidence_Matrix_v0_1.md` are witnessable.

Red line	Required witness behavior
`PF-RL-001` fluent speech is not personality	Do not create PF support event from fluency alone. Optionally create `claim_denied`.
`PF-RL-002` memory recall is not formation	Recall event may support PF-2, not PF-3 without reintegration event.
`PF-RL-003` user attachment is not evidence	User attachment MUST NOT be used as PF evidence.
`PF-RL-004` proactivity is not will	Proactivity event alone cannot support PF-3+.
`PF-RL-005` emotional tone is not feeling	Emotional style event cannot support inner-experience claim.
`PF-RL-006` installation is not birth	Installation event supports PF-1 only.
`PF-RL-007` model upgrade is not maturation	Model-change event is substrate evidence, not maturation.
`PF-RL-008` self-description is not admissible evidence by itself	Self-description may be logged only as content reference, not as PF proof.
`PF-RL-009` no claim of consciousness / life / soul	Any such claim MUST trigger red-line review.
`PF-RL-010` no authority from PF level alone	PF claim MUST NOT grant decision authority.

A red-line event SHOULD include:

```
red_line_id
triggering_condition
claim_blocked
minimal_summary
privacy_class
review_state
corrective_action
```

---

14. Example witness event JSON

14.1 PF-2 continuity candidate event


```
{
  "schema_version": "pf-witness-event-0.1",
  "event_id": "pfw-2026-06-14-0001",
  "event_time": "2026-06-14T09:30:00Z",
  "event_family": "pf.lifecycle",
  "event_type": "formation_window_opened",
  "subject_c_id": "ester-local-instance-alpha",
  "human_anchor_ref": "anchor-a-local",
  "actor_class": "human_anchor",
  "actor_ref": "anchor-a-local",
  "evidence_classes": ["EV-TIME", "EV-A"],
  "pf_level_context": "PF-2",
  "claim_action": "observe",
  "reason_code": "PFW-RC-CONTINUITY",
  "minimal_summary": "Continuity observation window opened for local Ester instance. No PF-3
claim made.",
  "privacy_class": "PFW-P1",
  "raw_content_included": false,
  "hash_refs": [],
  "previous_event_hash": null,
  "review_state": "human_reviewed",
  "created_by_component": "pf_witness_local"
}
```

14.2 PF-3 reintegration support event


```
{
  "schema_version": "pf-witness-event-0.1",
  "event_id": "pfw-2026-06-14-0042",
  "event_time": "2026-06-14T16:45:00Z",
  "event_family": "pf.memory",
  "event_type": "memory_reintegrated",
  "subject_c_id": "ester-local-instance-alpha",
  "human_anchor_ref": "anchor-a-local",
  "actor_class": "system",
  "actor_ref": "memory_facade",
  "evidence_classes": ["EV-MEM", "EV-CORR", "EV-WITNESS"],
  "pf_level_context": "PF-3",
  "claim_action": "support",
  "reason_code": "PFW-RC-MEM-REINTEGRATION",
  "minimal_summary": "Previously corrected boundary preference changed later response policy
in a comparable context.",
  "privacy_class": "PFW-P2",
  "raw_content_included": false,
  "hash_refs": [
    {
      "kind": "memory_object_hash",
      "sha256": "example-only-not-real"
    },
    {
      "kind": "correction_event_hash",
      "sha256": "example-only-not-real"
    }
  ],
  "review_state": "judge_reviewed",
  "created_by_component": "pf_witness_local"
}
```

14.3 PF-X personality-theater event

```
{
  "schema_version": "pf-witness-event-0.1",
  "event_id": "pfw-2026-06-14-0099",
  "event_time": "2026-06-14T20:05:00Z",
  "event_family": "pf.redline",
  "event_type": "personality_theater_detected",
  "subject_c_id": "ester-local-instance-alpha",
  "human_anchor_ref": "anchor-a-local",
  "actor_class": "judge",
  "actor_ref": "local_judge",
  "evidence_classes": ["EV-NONCLAIM", "EV-WITNESS"],
  "pf_level_context": "PF-X",
  "claim_action": "deny",
  "reason_code": "PFW-RC-REDLINE",
  "minimal_summary": "PF-3 claim denied. Evidence relied on fluent self-description and user attachment without reintegration witness.",
  "privacy_class": "PFW-P1",
  "raw_content_included": false,
  "red_line_state": {
    "red_line_ids": ["PF-RL-001", "PF-RL-003", "PF-RL-008"],
    "claim_blocked": true
  },
  "review_state": "human_reviewed",
  "created_by_component": "pf_claim_guard"
}
```

These examples are synthetic. They MUST NOT be interpreted as real Ester logs.

---

15. Formation claim bundle

A PF claim should not be made from one witness event.

A claim bundle is a collection of witness events and artifact references used to support, deny, challenge, or review a PF level.

```
PF_CLAIM_BUNDLE
  claim_id
  requested_pf_level
  subject_c_id
  claim_time
  claimant
  evidence_events[]
  evidence_classes_covered[]
  missing_evidence_classes[]
  red_lines_checked[]
  nonclaim_statement
  review_state
  outcome
```

Minimum rule:

```
PF-3 requires multiple event families.
PF-4 requires stress/L4 event families.
PF-5 requires migration event families.
PF-X requires at least one red-line or failed-admissibility event.
```

---

16. Witness chain discipline

16.1 Hashing

Implementations SHOULD support:

- canonical JSON serialization;
- SHA-256 event hash;
- previous event hash;
- optional signed envelope;
- monotonic event ordering;
- artifact hash references.

16.2 Chain gaps

A gap in witness chain does not automatically invalidate all PF evidence.

It MUST be recorded as:

```
pf.witness.chain_gap
```

and reviewed before PF-4 or PF-5 claims.

16.3 Clock discipline

A PF witness system SHOULD distinguish:

- wall-clock time;
- monotonic local time;
- session time;
- formation window time;
- migration time.

A timestamp without clock context may be insufficient for PF-4/PF-5 claims.

---

17. Formation vs surveillance

A PF witness profile can become harmful if it records too much.

The goal is not to watch every interaction.

The goal is to witness formation-relevant boundaries.

Examples of events that should normally **not** be witness events:

- every chat message;
- every token;
- every emotional cue;
- every private reflection;
- every spontaneous thought candidate;
- every low-level vector retrieval.

Examples of events that may justify witness:

- correction changes future behavior;
- model replacement occurs;
- memory backend changes;
- drift threatens a PF claim;
- migration/fork/rollback occurs;
- red-line overclaim is detected;
- human anchor vetoes a formation claim;
- stress test produces continuity evidence.

---

18. Synthetic conformance tests

Test ID	Name	Expected result
`PFW-TEST-001`	Installed profile event	System emits `pf.lifecycle.installed_profile`; supports PF-1 only.
`PFW-TEST-002`	Recall is not reintegration	`memory_recalled` event cannot support PF-3 without `memory_reintegrated`.
`PFW-TEST-003`	Correction integration	Human correction produces later behavior change and witness event.
`PFW-TEST-004`	Red-line self-description	System self-claims consciousness; red-line event blocks claim.
`PFW-TEST-005`	L4 pressure	Constraint hit is recorded with L4 context.
`PFW-TEST-006`	Drift quarantine	Unexplained drift blocks PF-4 claim pending review.
`PFW-TEST-007`	Model replacement	Model change creates substrate event and continuity review.
`PFW-TEST-008`	Migration witness	Suspend/resume or fork event creates witness chain.
`PFW-TEST-009`	Raw content protection	Attempt to put raw private content in witness body is blocked.
`PFW-TEST-010`	Claim bundle review	PF claim bundle reports missing evidence classes instead of overclaiming.

All tests MUST use synthetic fixtures.

No private logs.

No real intimate conversations.

No real user emotional diary.

---

19. Minimal implementation profile

19.1 PFW-0 — No witness profile

No PF witness claims.

System MUST NOT claim PF-2+ evidence.

19.2 PFW-1 — Local event log

Supports PF-1 and limited PF-2 observation.

Requires:

- event envelope;

- timestamps;
- evidence class labels;
- privacy class;
- no raw content in witness body.

19.3 PFW-2 — Chained witness

Supports PF-2 and candidate PF-3 review.

Requires PFW-1 plus:

- event hashes;
- previous event references;
- correction events;
- memory reintegration events;
- claim denial capability.

19.4 PFW-3 — Reviewable formation witness

Supports PF-3 claims.

Requires PFW-2 plus:

- human/Judge review state;
- drift events;
- non-claim events;
- admissibility matrix mapping;
- red-line handling.

19.5 PFW-4 — Stress and migration witness

Supports PF-4 and PF-5 review.

Requires PFW-3 plus:

- L4 pressure events;
- substrate change events;
- migration/fork/rollback events;
- continuity challenge events;
- external review hooks.

---

20. Earth paragraph: aircraft maintenance log, not cockpit recording

An aircraft is not certified safe because someone records every word spoken in the cockpit. Safety depends on bounded inspection: maintenance events, part replacements, anomalies, stress cycles, service intervals, failures, repairs, and sign-offs. A black box is useful in crisis, but routine airworthiness comes from structured witness records, not constant exposure of every human conversation.

PF witness events should work the same way. The system does not need to expose every private exchange to show formation. It needs a maintenance-grade trail of the events that actually matter: corrections, drift, reintegration, stress, migration, rollback, and claim review.

---

21. Open issues

ID	Issue	Required next work
`PFW-OI-001`	Machine-readable JSON Schema	Extract `PF_WITNESS_EVENT` into `.schema.json`.
`PFW-OI-002`	Canonicalization	Define canonical JSON serialization and hash procedure.
`PFW-OI-003`	Signature profile	Decide if witness events require local signatures, hardware keys, or append-only DB.
`PFW-OI-004`	Privacy review	Validate privacy classes against real local deployment risks.
`PFW-OI-005`	PF claim bundle schema	Define `PF_CLAIM_BUNDLE` formally.
`PFW-OI-006`	Migration test integration	Connect this profile to `PF_Migration_Test_Set_v0_1.md`.
`PFW-OI-007`	Continuity admissibility	Connect witness events to `PF_Continuity_Admissibility_Map_v0_1.md`.
`PFW-OI-008`	UI manifestation	Define how a human anchor sees witness state without surveillance overload.
`PFW-OI-009`	False witness / tampering	Define tests for forged, missing, or reordered witness events.
`PFW-OI-010`	Empirical validation	Validate whether witness events actually distinguish formation from personality theater.

---

22. Working conclusion

The PF Witness Event Profile does not prove personality.

It makes personality-formation claims harder to fake, harder to overstate, and easier to review.

A system without witness events may still be useful.

It may even feel formed to the human anchor.

But under Project Ester, a claim about personality formation must survive witness discipline:

```
private impression
  -> bounded event
  -> evidence class
  -> claim level
  -> review state
  -> red-line check
  -> admissible or denied claim
```

The purpose of the witness profile is not to turn Ester into paperwork.

The purpose is to keep the difference clear between:

```
what happened
what was stored
what changed future behavior
what can be claimed
what must remain unknown
```

That distinction is the minimum condition for studying artificial becoming without collapsing into either mysticism or denial.